微型作業系統實作期末報告

myOSshell

第二十六組

資管三乙 11244213 鄭翔元

資管三乙 11244244 蘇奕安

目錄

目錄 2
緣起與目的 4
一、專案簡介 6
二、系統架構 7
2.1 程式檔案結構 7
2.2 核心資料結構 7
2.3 主流程 7
三、指令解析設計說明 9
3.1 判斷背景執行符號 & 9
3.2 依 Pipe 符號 | 切割成多個指令段 9
3.3 解析每一段指令的參數與重新導向符號 9
四、Process 執行設計說明 11
4.1 為什麼一定要用 fork() + execvp() 11
4.2 前景指令的執行流程 11
4.3 背景執行的差異 12
4.4 避免殭屍處理元（Zombie Process） 12
五、內建指令設計說明 14
5.1 cd 為何不能用 execvp() 14
5.2 pwd 與 exit 14
六、Pipe 管線設計說明 15
6.1 為什麼需要 pipe() 與 dup2() 15
6.2 多重 pipe 的處理方式 15
七、I/O Redirection 設計說明 17
7.1 輸出重新導向 > 17
7.2 追加輸出 >> 17
7.3 輸入重新導向 < 17
八、錯誤處理設計說明 18
九、系統測試與驗證 19
9.1 編譯與啟動 19
9.2 基本指令 19
9.3 指定 16 項指令 22

9.4 其他建議指令 25
9.5 背景執行測試 26
9.6 Pipe 測試 27
9.7 Redirection 測試 28
9.8 內建指令測試 30
9.9 錯誤處理測試 31
十、16 項指定指令功能說明 33
(1) ps：觀察系統所有的處理元資料 33
(2) ps axjf：連同部分處理元樹狀態顯示 33
(3) ps aux：顯示詳細 process 清單 33
(4) ls -l：長格式顯示檔案資訊 34
(5) ls -l | more：pipe 串接顯示 34
(6) top：即時系統資源監控 34
(7) cal：顯示月曆 34
(8) whoami：顯示目前使用者 34
(9) date：顯示日期時間 35
(10) pwd：顯示工作目錄 35
(11) mv：移動 / 重新命名檔案 35
(12) cp：複製檔案 35
(13) file：判斷檔案類型 35
(14) cat：顯示檔案內容 35
(15) rm：刪除檔案 36
(16) vi：執行原始代碼全螢幕編輯 36
十一、開發過程中遇到的困難與解決方式 37
11.1 指令切割與參數解析 37
11.2 fork() 後 parent 與 child 的差異 37
11.3 為什麼 cd 不能只用 execvp() 37
11.4 Pipe 的檔案描述符管理 37
11.5 背景執行需避免殭屍處理元 38
11.6 Redirection 需要 open() 與 dup2() 搭配使用 38
十二、結論 39
十三、附錄：使用到的系統呼叫一覽 40

緣起與目的

要能完全理解作業系統，最好的方法就是實作出一個作業系統，因此本專案選擇以「微型作業系統實作」作為期末專題的方向。電腦硬體組裝完成之後，要讓應用程式可以運行，關鍵就是作業系統：一個程式編寫完成之後，須經過編譯（compile）成為目的碼（object code），再連結（link）其他程式庫（library），變成機器碼（machine code），最後才裝入（load）記憶體（memory）運行。

在這整個流程中，shell 在 UNIX 或 Linux 作業系統中是一種非常重要的使用者介面，可以透過命令列的方式輸入指令，與作業系統進行互動。本專案透過作業系統原型的實作，具體寫出 shell 的操作方式——並不只是單純呼叫系統既有的指令，而是像開發一套作業系統一樣，從零建立資料結構，逐步實作出作業系統應有的功能。

使用者可以在命令列的提示字元（$ 或 >）後方輸入指令，也可以在指令後方加上 & 符號要求背景執行，這代表父處理元（parent process）與子處理元（child process）將以並行（concurrently）的方式執行，不需要等待彼此。而在程式設計的應用情境上，也可以使用像 vi 這樣的指令，進行全螢幕的程式碼編輯。例如，本專案完成後的程式設計結果，可以執行如下：

```
myOSshell $ cat myshell.c
```

或是以背景方式執行：

```
myOSshell $ cat myshell.c &
```

另外，下列幾個指令也都能在本專案的 myOSshell $ 提示字元底下正常運行：

- vi：編輯代碼
- exit：結束程式執行
- ls：顯示目錄

- mkdir：創建目錄
- rmdir：刪除目錄

使用 shell 功能部分的系統實際執行畫面，詳見本報告第九章「系統測試與驗證」第 9.2 節（基本指令測試，圖 9-1 至圖 9-3）。必須先從作業系統的資料結構建構好，再持續進行程式設計，執行命令後將結果完整顯示列出，如此才能真正理解作業系統是如何運作的。

當完成這個專案之後，運行程式即可顯示提示字元 myOSshell $（本實作採用 $ 作為提示符號），接著可以下達各種命令，包括 vi 指令等，並有以下幾點補充說明：

1. vi：執行原始代碼全螢幕編輯。
2. 程式中使用 fork() 可以啟動新的處理元。
3. 這些 shell 指令主要是在 UNIX 或 Linux 作業系統上運行；如果在 Windows 環境下運行，除非 Windows 作業系統完整支援，否則可能會發生程式出錯或 shell 指令無法運行的情況。本專案實際是在 Windows 搭配 WSL（Windows Subsystem for Linux）的 Ubuntu 環境下開發與測試，以確保所有指令都能在真正的 Linux 核心上正確執行。

以下這 16 項指令，依老師要求逐一填上指令功能說明：ps 與 ls 兩個指令為必須實作項目，可獲得 60 分基本分，其餘指令皆已盡可能完整實作，詳細的功能說明列於第十章，對應的實際測試畫面與 file 功能的系統截圖則統一放在第九章「系統測試與驗證」中呈現，以便對照 test_cases.md 的測試流程逐項檢視。

一、專案簡介

myOSshell 是一個以 C 語言實作的微型 Linux Shell。使用者執行程式後，畫面會顯示自訂提示字元 myOSshell $，使用者可在提示字元後輸入 UNIX / Linux 指令，Shell 會解析輸入內容，並透過建立子處理元（child process）的方式實際執行該指令，而不是單純呼叫系統的終端機或把指令結果寫死在程式裡。

本專案的目的並非「做出一個看起來像 Shell 的介面」，而是要透過實作的過程，具體理解作業系統中幾個核心概念：

1. 命令列使用者介面如何讀取與顯示
2. 指令字串如何被拆解、解析成可執行的格式
3. 父處理元（parent process）與子處理元（child process）的關係
4. fork() 如何複製出一個新的處理元
5. execvp() 如何讓子處理元載入並執行外部程式
6. waitpid() 如何讓父處理元等待子處理元執行完畢
7. 背景執行（&）如何讓父子處理元並行運作
8. Pipe 管線（|）如何透過檔案描述符（file descriptor）讓兩個處理元串接資料
9. 輸入 / 輸出重新導向（<、>、>>）如何改變一個處理元的標準輸入輸出來源
10. 一個健全的系統程式應該如何處理各種不正常的輸入，而不會直接崩潰

二、系統架構

2.1 程式檔案結構

本專案依照功能拆分為三個原始碼檔案與一個共用標頭檔，刻意避免把所有邏輯塞在 main() 函式裡：

```
myshell.h     -- 定義共用資料結構（Command、ParsedLine）與所有函式的宣告
myshell.c     -- 主迴圈、提示字元顯示、讀取使用者輸入、SIGCHLD 訊號處理
parser.c      -- 將使用者輸入的字串解析成可執行的指令結構
executor.c    -- 內建指令、fork/execvp/waitpid、管線與重新導向的實際執行邏輯
```

2.2 核心資料結構

Shell 在解析完一行輸入後，會把結果存成以下兩種資料結構：

```
typedef struct {
    char *argv[MAX_ARGS];   // 指令本身與所有參數，例如 {"ls", "-l", NULL}
    char *input_file;       // 若有 < 重新導向，記錄輸入檔名
    char *output_file;      // 若有 > 或 >> 重新導向，記錄輸出檔名
    int   append_output;    // 1 表示 >>（追加），0 表示 >（覆蓋）
} Command;

typedef struct {
    Command commands[MAX_COMMANDS]; // 一行輸入可能因 | 而拆成多個指令
    int command_count;              // 這一行總共有幾個指令（pipe 段數）
    int background;                 // 是否以 & 結尾，要求背景執行
} ParsedLine;
```

這樣設計的理由是：一行使用者輸入（例如 ps aux | grep root | more）實際上代表「多個指令」串接在一起，所以用 ParsedLine 包住一個或多個 Command，可以同時處理「單一指令」與「多重 pipe」兩種情況，不需要另外寫一套邏輯。

2.3 主流程

整體執行流程如下：

```
顯示提示字元 myOSshell $
        ↓
讀取使用者輸入一整行（fgets）
```

```
        ↓
若為空白輸入 → 直接回到顯示提示字元
        ↓
解析指令字串（parse_line）
  → 拆出 command / 參數 / pipe 段 / & / < / > / >>
  → 若語法有誤 → 顯示錯誤訊息 → 回到顯示提示字元
        ↓
判斷是否為內建指令（cd / pwd / exit）
  → 是 → 在 Shell 自己的處理元中直接執行 → 回到顯示提示字元
        ↓
建立子處理元執行外部指令（execute_pipeline）
  → fork() 建立一個或多個 child process
  → 每個 child process 呼叫 execvp() 執行對應指令
  → 若有 pipe，先用 pipe() + dup2() 把前一個指令的輸出接到下一個指令的輸入
  → 若有 < / > / >>，用 open() + dup2() 重新導向 stdin / stdout
        ↓
判斷是否為背景執行
  → 是 → 父處理元立即回到提示字元，不等待子處理元
  → 否 → 父處理元呼叫 waitpid() 等待子處理元結束
        ↓
回到顯示提示字元
```

實際測試時 Shell 啟動與基本指令的執行畫面，詳見第九章圖 9-1、圖 9-2、圖 9-3。

三、指令解析設計說明

使用者輸入的一行文字（例如 ls -l | more）必須先被拆解成程式可以理解的結構，這個工作由 parser.c 中的 parse_line() 負責，主要分為三個步驟：

3.1 判斷背景執行符號 &

先檢查整行字串去除前後空白後，最後一個字元是否為 &。若是，標記 background = 1，並把這個符號從字串中移除，再繼續處理剩下的部分。

例如輸入 sleep 10 &，解析後會得到：

```
command = sleep 10
background = true
```

3.2 依 Pipe 符號 | 切割成多個指令段

使用 strtok_r() 以 | 為分隔符號，把整行字串切成多段。每一段代表一個獨立的指令，並依序存入 ParsedLine.commands[] 陣列中。本實作支援多重 pipe，例如 ps aux | grep root | more 會被拆成三段：

```
command 1 = ps aux
command 2 = grep root
command 3 = more
```

切割時也會檢查 | 前後是否缺少指令（例如 ls | 或 | more），若偵測到語法錯誤會立即顯示 myOSshell: syntax error 並中止解析，不會讓程式繼續用不完整的資料去執行。

3.3 解析每一段指令的參數與重新導向符號

每一個指令段再交給 parse_segment() 逐字元掃描，依序判斷：

- 遇到空白：跳過，作為參數之間的分隔
- 遇到 <：取後面接續的檔名做為輸入檔案

- 遇到 >：再檢查下一個字元是否也是 >，藉此區分「覆蓋輸出 >」與「追加輸出 >>」，並取後面接續的檔名做為輸出檔案
- 其他情況：視為一般的指令名稱或參數，依序填入 argv[] 陣列

例如輸入 ps axjf，解析結果為：

```
command = ps
argv[0] = ps
argv[1] = axjf
```

若 <、>、>> 後面沒有接檔名（例如 ls > ），會立即回報 myOSshell: missing output file 或 missing input file，不會把空字串當作檔名去呼叫 open()。實際測試畫面詳見第九章圖 9-14。

四、Process 執行設計說明

4.1 為什麼一定要用 fork() + execvp()

fork() 是 Linux 中用來建立一個新處理元的系統呼叫。呼叫後，目前的處理元會被複製成兩份：一份是原來的處理元（稱為 parent process），另一份是幾乎完全相同的複製品（稱為 child process）。兩者唯一的差別是 fork() 的回傳值——在 parent process 中回傳子處理元的 PID，在 child process 中回傳 0。

Shell 之所以必須要用 fork()，是因為如果不複製出一個新的處理元，而是直接在自己的處理元裡呼叫 execvp()，那麼目前正在執行的 myOSshell 程式本身就會被「換掉」，變成執行 ls，ls 執行完程式就結束了，根本回不到 myOSshell $ 提示字元。

所以正確的作法是：parent process（也就是 Shell 本身）呼叫 fork() 複製出一個 child process，由 child process 去呼叫 execvp() 把自己換成要執行的指令（例如 ls），執行完後 child process 結束，而 parent process 完全沒有受到影響，可以繼續顯示提示字元、等待下一次輸入。

4.2 前景指令的執行流程

以使用者輸入 ls 為例，完整流程為：

1. Shell（parent process）呼叫 parse_line() 解析出 argv = {"ls", NULL}
2. parent process 呼叫 fork()
3. child process（fork() 回傳值為 0 的那一份）呼叫 execvp("ls", argv)，把自己換成 ls 程式並執行

4. parent process（fork() 回傳值為子處理元 PID 的那一份）呼叫 waitpid(pid, &status, 0)，暫停自己的執行，直到指定的 child process 執行完畢
5. child process 執行完 ls 後自然結束，parent process 的 waitpid() 返回，程式回到 myOSshell $ 等待下一次輸入

實際以 ps、ps axjf、ps aux 進行測試，驗證上述流程確實正確運作，測試畫面詳見第九章圖 9-1、圖 9-2。

4.3 背景執行的差異

當使用者輸入指令並以 & 結尾（例如 sleep 10 &），流程與前景執行幾乎相同，唯一的差別在第 4 步：parent process 不會呼叫 waitpid() 等待，而是立即印出 [background pid: <子處理元 PID>]，然後直接回到提示字元接受下一個輸入。child process 則在背後繼續執行，與 parent process 並行（concurrent）運作。實際測試畫面詳見第九章圖 9-9。

4.4 避免殭屍處理元（Zombie Process）

當一個 child process 結束時，作業系統不會立刻把它從系統中完全移除，而是讓它變成「殭屍處理元」（zombie process），保留其結束狀態，直到 parent process 透過 wait() 或 waitpid() 把這個狀態「收回」為止。

對前景指令而言，因為 parent process 一定會呼叫 waitpid() 等待，所以不會產生殭屍處理元。但背景執行的指令結束時，parent process 正忙著等待使用者輸入下一行指令，不會主動去 waitpid() 那個背景子處理元，因此本實作額外利用 SIGCHLD 訊號處理機制解決這個問題：

```
static void sigchld_handler(int sig) {
    cleanup_background_processes();
}
```

每當任何一個子處理元結束時，作業系統會送出 SIGCHLD 訊號給 parent process，觸發

sigchld_handler()，其內部呼叫：

```
waitpid(-1, &status, WNOHANG)
```

-1 代表「不指定特定 PID，誰結束了就回收誰」；WNOHANG 代表「如果目前沒有任何子處理元結束，不要卡住等待，直接返回」。這樣一來，背景處理元一結束就會立刻被回收，不會在系統中累積殭屍處理元。除了訊號處理之外，Shell 主迴圈每跑完一輪也會再呼叫一次 cleanup_background_processes() 作為雙重保險。

五、內建指令設計說明

exit、cd、pwd 這三個指令並不會透過 fork() + execvp() 執行，而是直接由 Shell 自己的處理元處理，原因如下：

5.1 cd 為何不能用 execvp()

如果 cd 也用 fork() 建立子處理元，再讓子處理元呼叫 chdir() 改變工作目錄，那麼改變的只會是「子處理元」的工作目錄。子處理元結束之後，這個改變會跟著子處理元一起消失，parent process（也就是 Shell 本身）的工作目錄完全不會受影響。使用者會發現不管打多少次 cd，pwd 顯示的路徑永遠不會改變。

因此 cd 必須在 Shell 自己的處理元中直接呼叫：

```
chdir(target_directory)
```

這樣才能真正改變 Shell 本身的工作目錄，後續輸入的相對路徑指令才會以新的目錄為基準。

5.2 pwd 與 exit

pwd 使用 getcwd() 取得目前工作目錄並印出；exit 則直接呼叫 exit(0) 結束整個 Shell 程式。這兩者雖然理論上也可以透過外部程式 /bin/pwd 執行，但為了精準對應「Shell 自己的狀態」，本實作選擇以內建方式處理。實際以 pwd、cd ..、cd /tmp、exit 進行測試，驗證 cd 確實改變了 Shell 本身的工作目錄，而非僅子處理元的工作目錄；測試畫面詳見第九章圖 9-13。

六、Pipe 管線設計說明

6.1 為什麼需要 pipe() 與 dup2()

ls -l | more 這種指令代表「把 ls -l 的標準輸出，接到 more 的標準輸入」。要達成這個效果，需要：

1. pipe(fds)：建立一個管線，回傳兩個檔案描述符，fds[0] 是讀取端，fds[1] 是寫入端，兩者中間是核心維護的一塊緩衝區
2. 建立第一個 child process 執行 ls -l，但在 execvp() 之前，先用 dup2(fds[1], STDOUT_FILENO) 把這個 child process 的標準輸出重新導向到管線的寫入端
3. 建立第二個 child process 執行 more，同樣在 execvp() 之前，用 dup2(fds[0], STDIN_FILENO) 把標準輸入重新導向到管線的讀取端
4. parent process 把自己手上沒用到的管線兩端都 close() 掉，避免檔案描述符洩漏，也避免讀取端因為寫入端沒關閉而一直等不到結束訊號
5. parent process 呼叫 waitpid() 等待兩個 child process 都結束

6.2 多重 pipe 的處理方式

對於 ps aux | grep root | more 這種有兩個 | 的情況，本實作會先建立 n-1 組管線（n 為指令段數），再依序為每一段指令建立一個 child process。每個 child process 依照自己在序列中的位置，判斷該把標準輸入接到哪一組管線的讀取端、標準輸出接到哪一組管線的寫入端：

- 第一段：只需要重新導向標準輸出（接到下一組管線）
- 中間段：標準輸入接前一組管線，標準輸出接下一組管線
- 最後一段：只需要重新導向標準輸入（接前一組管線）

所有 child process 建立完成後，parent process 會把所有管線的檔案描述符關閉，再依序 waitpid() 等待每一個 child process 結束。

本實作已實際測試 ls -l | more、ps aux | grep bash、ps aux | grep root | more 三種情境，包含單一 pipe 與多重 pipe，輸出結果皆正確，證明 Pipe 機制運作正常，測試畫面詳見第九章圖 9-10。

七、I/O Redirection 設計說明

重新導向的實作原理是：在 child process 呼叫 execvp() 之前，先用 open() 開啟對應的檔案，再用 dup2() 把開啟後的檔案描述符複製到標準輸入（fd 0）或標準輸出（fd 1）的位置，這樣 execvp() 執行的程式完全不需要知道發生了重新導向，它依然只是對 fd 0 / fd 1 做讀寫，但實際上資料來源或去向已經被換成檔案。

7.1 輸出重新導向 >

```
open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644)
```

O_CREAT 表示檔案不存在就建立；O_TRUNC 表示如果檔案已存在，開啟時清空原內容，達到「覆蓋」的效果。

7.2 追加輸出 >>

```
open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644)
```

差別僅在 O_APPEND 取代 O_TRUNC，每次寫入都會被加到檔案尾端，不會清空原本的內容。

7.3 輸入重新導向 <

```
open(filename, O_RDONLY)
```

若指定的檔案不存在，open() 會回傳 -1，此時本實作會顯示 myOSshell: <檔名>: no such file or directory，而不會讓 child process 帶著無效的檔案描述符繼續執行 execvp()。

本實作已實際測試 ls -l > output.txt（覆蓋寫入）、echo hello >> output.txt（追加寫入）、cat < output.txt（輸入重新導向）三種情境，驗證結果皆符合預期，測試畫面詳見第九章圖 9-11、圖 9-12。

八、錯誤處理設計說明

一個只能在「輸入完全正確」的前提下運作的 Shell，並不符合作業系統程式應有的穩定性要求。本實作針對以下情境都加上了對應的檢查與提示，確保 Shell 不會因為使用者的輸入錯誤而崩潰：

錯誤情境	處理方式
空白輸入（直接按 Enter）	不做任何處理，直接顯示下一次提示字元
輸入不存在的指令	execvp() 失敗後顯示 command not found
cd 到不存在的資料夾	chdir() 失敗後顯示 no such file or directory
pipe 左邊或右邊缺指令（ls |、| more）	解析階段偵測到後顯示 syntax error
>、>>、< 後面缺檔名	解析階段偵測到後顯示 missing output/input file
重新導向指定的檔案不存在（< 情況）	open() 失敗後顯示對應錯誤訊息
fork() 失敗	顯示 fork failed
pipe() 失敗	顯示 pipe failed

這些檢查大多在「解析階段」就先攔截掉明顯的語法錯誤（例如缺檔名、pipe 兩端缺指令），避免帶著不完整或無效的資料進入後續的 fork() / execvp() 流程；而「執行階段」的錯誤（例如指令不存在、檔案不存在）則是透過檢查系統呼叫的回傳值來偵測，並用 perror() 或自訂訊息回報，絕不放任程式以未定義的狀態繼續執行。

本實作已實際測試全部 7 種錯誤輸入情境（不存在的指令、cd 到不存在的資料夾、pipe 兩端缺指令、重新導向缺檔名、重新導向的檔案不存在），Shell 在每一種情況下皆正確顯示對應錯誤訊息且未崩潰，測試畫面詳見第九章圖 9-14。

九、系統測試與驗證

本章依照 test_cases.md 測試紀錄文件所列的九大測試類別，逐項呈現實際執行的測試畫面與結果說明。所有測試均於 Ubuntu（WSL2）環境下，先以 make 完成編譯，再逐項手動輸入指令進行驗證。

9.1 編譯與啟動

執行 make 進行編譯，再以 ./myOSshell 啟動本專案的 Shell。編譯完成後畫面顯示自訂提示字元 myOSshell $，即可開始輸入指令進行後續測試，過程與 test_cases.md 中的預期輸出一致。

9.2 基本指令

測試 ls（顯示目前目錄檔案）、ps（顯示目前 process）兩個基本指令是否能正確透過 fork() + execvp() 執行。以下為 ps 與 ps aux 的測試結果：ps 顯示目前終端機下的處理元（bash、myOSshell、ps 本身），ps aux 則列出系統中所有使用者的詳細處理元資訊：

```
myOSshell $ ps
    PID TTY          TIME CMD
   1711 pts/2    00:00:00 bash
   1895 pts/2    00:00:00 myOSshell
   1925 pts/2    00:00:00 ps
myOSshell $ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.1  21748 12508 ?        Ss   18:04   0:01 /sbin/ini
root           2  0.0  0.0   3072  1792 ?        Sl   18:04   0:00 /init
root           6  0.0  0.0   3072  1792 ?        Sl   18:04   0:00 plan9 --c
root          42  0.0  0.1  50436 15360 ?        S<s  18:04   0:00 /usr/lib/
root          88  0.0  0.0  25276  6400 ?        Ss   18:04   0:00 /usr/lib/
systemd+     113  0.0  0.1  21460 12416 ?        Ss   18:04   0:00 /usr/lib/
systemd+     115  0.0  0.0  91028  7424 ?        Ssl  18:04   0:00 /usr/lib/
root         173  0.0  0.0   4236  2560 ?        Ss   18:04   0:00 /usr/sbin
message+     174  0.0  0.0   9624  4736 ?        Ss   18:04   0:00 @dbus-dae
root         181  0.0  0.1  18160  8448 ?        Ss   18:04   0:00 /usr/lib/
root         190  0.0  0.0   3160  1920 hvc0     Ss+  18:04   0:00 /sbin/age
syslog       202  0.0  0.0 222508  5376 ?        Ssl  18:04   0:00 /usr/sbin
root         205  0.0  0.0   3116  1792 tty1     Ss+  18:04   0:00 /sbin/age
root         214  0.0  0.2 107024 22516 ?        Ssl  18:04   0:00 /usr/bin/
root         305  0.0  0.0   6824  4352 pts/1    Ss   18:04   0:00 /bin/logi
xnikz        348  0.0  0.1  20312 11264 ?        Ss   18:04   0:00 /usr/lib/
xnikz        349  0.0  0.0  21156  3520 ?        S    18:04   0:00 (sd-pam)
xnikz        373  0.0  0.0   6072  5248 pts/1    S+   18:04   0:00 -bash
root         407  0.0  0.0   3100  1032 ?        Ss   18:04   0:00 /init
root         408  0.0  0.0   3116  1172 ?        S    18:04   0:00 /init
xnikz        409  0.0  0.0   4884  3328 pts/0    Ss+  18:04   0:00 bash -lc
root         415  0.0  0.0  13760  6400 pts/0    S+   18:04   0:00 sudo apt-
xnikz        416  0.0  0.0   3156  1664 pts/0    S+   18:04   0:00 tail -5
root         792  0.0  0.0   6668  4480 pts/4    Ss   18:35   0:00 /bin/logi
root         837  0.0  0.1  20116 11008 ?        Ss   18:35   0:00 /usr/lib/
root         838  0.0  0.0  21164  3376 ?        S    18:35   0:00 (sd-pam)
root         860  0.0  0.0   6072  5120 pts/4    S+   18:35   0:00 -bash
polkitd     1273  0.0  0.0 308164  7808 ?        Ssl  18:37   0:00 /usr/lib/
root        1709  0.0  0.0   3088  1032 ?        Ss   19:05   0:00 /init
root        1710  0.0  0.0   3088  1168 ?        S    19:05   0:00 /init
xnikz       1711  0.0  0.0   6072  5248 pts/2    Ss   19:05   0:00 -bash
root        1873  0.0  0.1 1756620 12672 ?       Ssl  20:08   0:00 /usr/libe
xnikz       1895  0.0  0.0   2684  1536 pts/2    S+   20:09   0:00 ./myOSshe
root        1926  0.0  0.0  25280  3596 ?        S    20:15   0:00 (udev-wor
```

圖 9-1：執行 ps 顯示目前處理元，再執行 ps aux 列出所有使用者的詳細處理元資訊

再以 ps axjf 測試樹狀格式輸出，確認 myOSshell 與其子處理元之間的父子關係能以樹狀結構正確呈現：

```
myOSshell $ ps axjf
   PPID     PID    PGID     SID TTY        TPGID STAT   UID   TIME COMMAND
      0       1       1       1 ?           -1 Ss       0   0:01 /sbin/ini
      1       2       0       0 ?           -1 Sl       0   0:00 /init
      2       6       0       0 ?           -1 Sl       0   0:00  \_ plan9
      2     305     305     305 pts/1       373 Ss       0   0:00  \_ /bin/
    305     373     373     305 pts/1       373 S+    1000   0:00  |   \_ -
      2     407     407     407 ?           -1 Ss       0   0:00  \_ /init
    407     408     407     407 ?           -1 S        0   0:00  |   \_ /
    408     409     409     409 pts/0       409 Ss+   1000   0:00  |      b
    409     415     409     409 pts/0       409 S+       0   0:00  |      s
    409     416     409     409 pts/0       409 S+    1000   0:00  |      t
      2     792     792     792 pts/4       860 Ss       0   0:00  \_ /bin/
    792     860     860     792 pts/4       860 S+       0   0:00  |   \_ -
      2    1709    1709    1709 ?           -1 Ss       0   0:00  \_ /init
   1709    1710    1709    1709 ?           -1 S        0   0:00      \_ /
   1710    1711    1711    1711 pts/2      1895 Ss    1000   0:00          -
   1711    1895    1895    1711 pts/2      1895 S+    1000   0:00          .
   1895    1929    1895    1711 pts/2      1895 R+    1000   0:00          p
      1      42      42      42 ?           -1 S<s      0   0:00 /usr/lib/
      1      88      88      88 ?           -1 Ss       0   0:00 /usr/lib/
      1     113     113     113 ?           -1 Ss     991   0:00 /usr/lib/
      1     115     115     115 ?           -1 Ssl    996   0:00 /usr/lib/
      1     173     173     173 ?           -1 Ss       0   0:00 /usr/sbin
      1     174     174     174 ?           -1 Ss     101   0:00 @dbus-dae
      1     181     181     181 ?           -1 Ss       0   0:00 /usr/lib/
      1     190     190     190 hvc0       190 Ss+      0   0:00 /sbin/age
      1     202     202     202 ?           -1 Ssl    102   0:00 /usr/sbin
      1     205     205     205 tty1       205 Ss+      0   0:00 /sbin/age
      1     214     214     214 ?           -1 Ssl      0   0:00 /usr/bin/
      1     348     348     348 ?           -1 Ss    1000   0:00 /usr/lib/
    348     349     348     348 ?           -1 S     1000   0:00  \_ (sd-p
      1     837     837     837 ?           -1 Ss       0   0:00 /usr/lib/
    837     838     837     837 ?           -1 S        0   0:00  \_ (sd-p
      1    1273    1273    1273 ?           -1 Ssl    990   0:00 /usr/lib/
      1    1873    1873    1873 ?           -1 Ssl      0   0:00 /usr/libe
myOSshell $
```

圖 9-2：執行 ps axjf，以樹狀格式顯示處理元之間的父子關係

接著測試 ls 與 ls -l：ls 列出目前目錄下的所有檔案與資料夾，ls -l 則以長格式顯示每個檔案的權限、擁有者、大小與修改時間：

```
myOSshell $ ls
Makefile    executor.o  myshell.h  old.txt     parser.c     test_cases.md
README.md   myOSshell   myshell.o  out.txt     parser.o
executor.c  myshell.c   new.txt    output.txt  screenshots
myOSshell $ ls -l
total 140
-rwxr-xr-x 1 xnikz xnikz   287 Jun 19 19:04 Makefile
-rwxr-xr-x 1 xnikz xnikz  3243 Jun 19 19:04 README.md
-rwxr-xr-x 1 xnikz xnikz  4646 Jun 19 19:04 executor.c
-rw-r--r-- 1 xnikz xnikz 15808 Jun 19 19:04 executor.o
-rwxr-xr-x 1 xnikz xnikz 34640 Jun 19 19:04 myOSshell
-rwxr-xr-x 1 xnikz xnikz  1431 Jun 19 19:04 myshell.c
-rwxr-xr-x 1 xnikz xnikz   829 Jun 19 19:04 myshell.h
-rw-r--r-- 1 xnikz xnikz 13320 Jun 19 19:04 myshell.o
-rw-r--r-- 1 xnikz xnikz    13 Jun 19 18:40 new.txt
-rw-r--r-- 1 xnikz xnikz     7 Jun 19 19:10 old.txt
-rw-r--r-- 1 xnikz xnikz   612 Jun 19 17:56 out.txt
-rw-r--r-- 1 xnikz xnikz   880 Jun 19 19:11 output.txt
-rwxr-xr-x 1 xnikz xnikz  4809 Jun 19 19:04 parser.c
-rw-r--r-- 1 xnikz xnikz 13584 Jun 19 19:04 parser.o
drwxr-xr-x 2 xnikz xnikz  4096 Jun 19 17:56 screenshots
-rwxr-xr-x 1 xnikz xnikz  3170 Jun 19 19:04 test_cases.md
myOSshell $
```

圖 9-3：執行 ls 列出目前目錄檔案，再執行 ls -l 以長格式顯示詳細檔案資訊

9.3 指定 16 項指令

依老師指定的 16 項指令逐一測試。首先測試 ls -l | more（pipe 串接顯示）、cal（顯示月曆）、date（顯示日期時間）、pwd（顯示工作目錄）：

```
myOSshell $ ls -l | more
total 140
-rwxr-xr-x 1 xnikz xnikz   287 Jun 19 19:04 Makefile
-rwxr-xr-x 1 xnikz xnikz  3243 Jun 19 19:04 README.md
-rwxr-xr-x 1 xnikz xnikz  4646 Jun 19 19:04 executor.c
-rw-r--r-- 1 xnikz xnikz 15808 Jun 19 19:04 executor.o
-rwxr-xr-x 1 xnikz xnikz 34640 Jun 19 19:04 myOSshell
-rwxr-xr-x 1 xnikz xnikz  1431 Jun 19 19:04 myshell.c
-rwxr-xr-x 1 xnikz xnikz   829 Jun 19 19:04 myshell.h
-rw-r--r-- 1 xnikz xnikz 13320 Jun 19 19:04 myshell.o
-rw-r--r-- 1 xnikz xnikz    13 Jun 19 18:40 new.txt
-rw-r--r-- 1 xnikz xnikz     7 Jun 19 19:10 old.txt
-rw-r--r-- 1 xnikz xnikz   612 Jun 19 17:56 out.txt
-rw-r--r-- 1 xnikz xnikz   880 Jun 19 19:11 output.txt
-rwxr-xr-x 1 xnikz xnikz  4809 Jun 19 19:04 parser.c
-rw-r--r-- 1 xnikz xnikz 13584 Jun 19 19:04 parser.o
drwxr-xr-x 2 xnikz xnikz  4096 Jun 19 17:56 screenshots
-rwxr-xr-x 1 xnikz xnikz  3170 Jun 19 19:04 test_cases.md
myOSshell $ cal
     June 2026
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30

myOSshell $ date
Fri Jun 19 20:17:03 CST 2026
myOSshell $ pwd
/home/xnikz/myOSshell_project
myOSshell $
```

圖 9-4：依序執行 ls -l | more、cal、date、pwd

接著測試 mv（移動 / 重新命名）、cp（複製檔案）、file（判斷檔案類型）、cat（顯示檔案內容）、rm（刪除檔案）：先以 mv old.txt new.txt 將檔案重新命名，再以 cp new.txt copy.txt 複製出一份副本，用 file 判斷其類型為 ASCII 純文字檔，cat 顯示其內容，最後以 rm 刪除該檔案：

```
myOSshell $ mv old.txt new.txt
myOSshell $ cp new.txt copy.txt
myOSshell $ file copy.txt
copy.txt: ASCII text
myOSshell $ cat copy.txt
"test"
myOSshell $ rm copy.txt
myOSshell $
```

圖 9-5：依序執行 mv 改名、cp 複製、file 判斷類型、cat 顯示內容、rm 刪除

再測試 top（即時系統資源監控）：top 會接管整個終端機畫面，即時更新處理元的 CPU、記憶體使用狀況，下圖為按下 q 離開前的處理元清單畫面，離開後正確回到 myOSshell $：

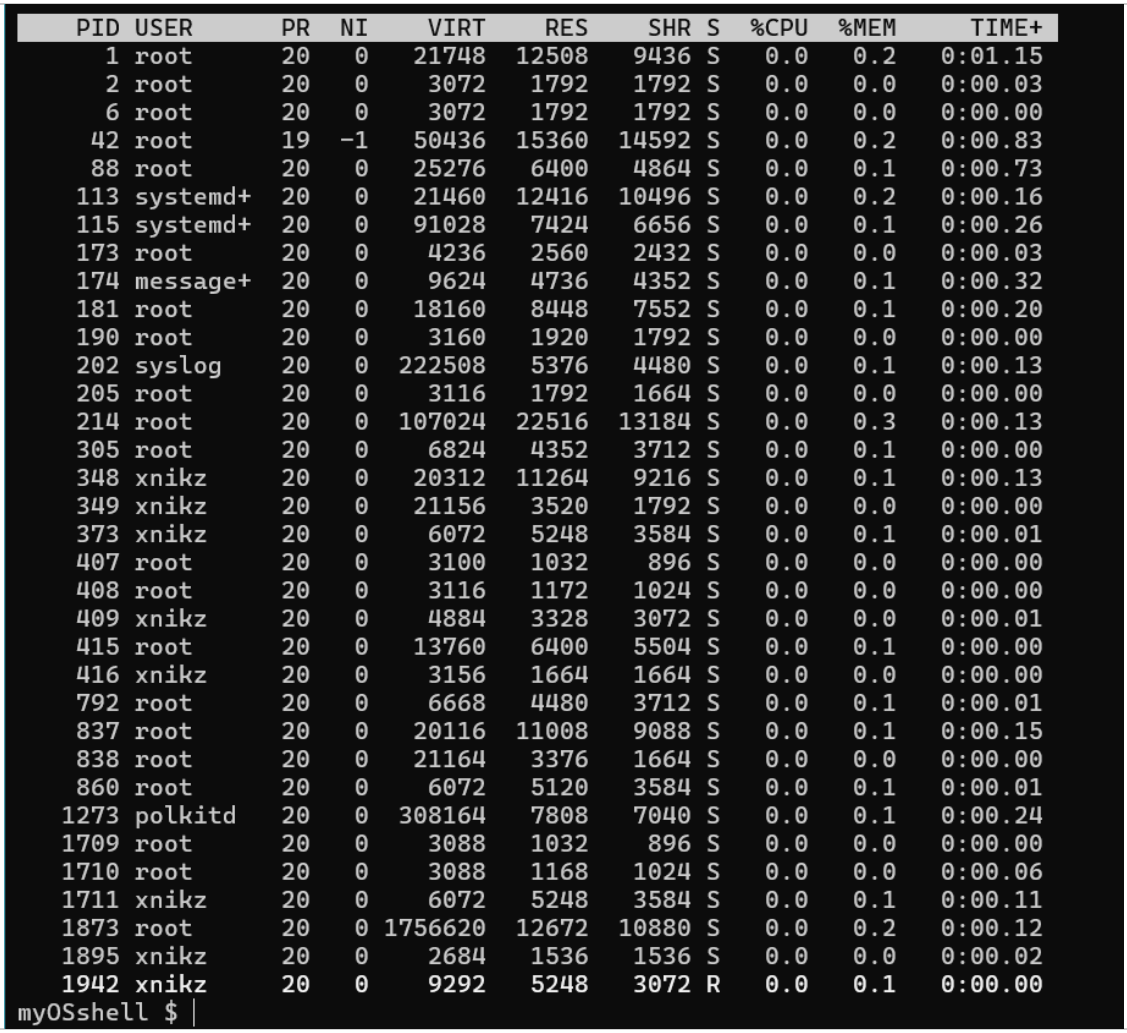


圖 9-6：執行 top，即時顯示系統處理元的資源使用狀況

最後測試 vi（全螢幕文字編輯器）：以 vi myshell.c 開啟本專案原始碼，畫面確實進入 vi 的全螢幕編輯模式（左下角顯示檔名與檔案大小，右下角顯示游標位置），證明 myOSshell 能正確把終端機控制權交給這類全螢幕程式：

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include "myshell.h"

void print_prompt(void) {
    printf("myOSshell $ ");
    fflush(stdout);
}

char *read_input(void) {
    static char line[MAX_LINE];
    if (fgets(line, sizeof(line), stdin) == NULL) {
        return NULL; /* EOF, e.g. Ctrl+D */
    }
    size_t len = strlen(line);
    if (len > 0 && line[len - 1] == '\n') {
        line[len - 1] = '\0';
    }
    return line;
}

static void sigchld_handler(int sig) {
    (void)sig;
    cleanup_background_processes();
}

int main(void) {
    struct sigaction sa;
    sa.sa_handler = sigchld_handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = SA_RESTART | SA_NOCLDSTOP;
    sigaction(SIGCHLD, &sa, NULL);

    while (1) {
        print_prompt();

"myshell.c" 67L, 1431B                                    5,18          Top
```

圖 9-7：執行 vi myshell.c，成功進入全螢幕編輯模式並顯示原始碼內容

輸入 :q 離開 vi 後，畫面正確返回 myOSshell $ 提示字元，行為與直接在系統終端機下執行 vi 完全一致。至此，老師指定的 16 項指令（ps、ps axjf、ps aux、ls -l、ls -l | more、top、cal、whoami、date、pwd、mv、cp、file、cat、rm、vi）已全數完成測試，詳細的指令功能說明請見第十章。

9.4 其他建議指令

除了指定的 16 項指令外，也額外測試了 mkdir（建立資料夾）、rmdir（刪除資料夾）、echo

（顯示文字）、grep（搜尋文字）。以下示範先以 mkdir testdir 建立資料夾、rmdir testdir 將其刪除，再以 echo hello 印出文字，最後以 grep main myshell.c 搜尋原始碼中含有 main 字串的那一行（int main(void) { ）：

```
myOSshell $ mkdir testdir
myOSshell $ rmdir testdir
myOSshell $ echo hello
hello
myOSshell $ grep main myshell.c
int main(void) {
myOSshell $
```

圖 9-8：依序執行 mkdir、rmdir、echo、grep main myshell.c

9.5 背景執行測試

測試 sleep 10 & 搭配 ps：輸入後 myOSshell 立即印出 [background pid: 1967] 並回到提示字元，沒有等待 sleep 指令的 10 秒鐘；緊接著執行 ps，可在處理元清單中看到 PID 1967 的 sleep 處理元仍在執行中，證明父子處理元確實以並行（concurrent）方式運作，而非依序等待：

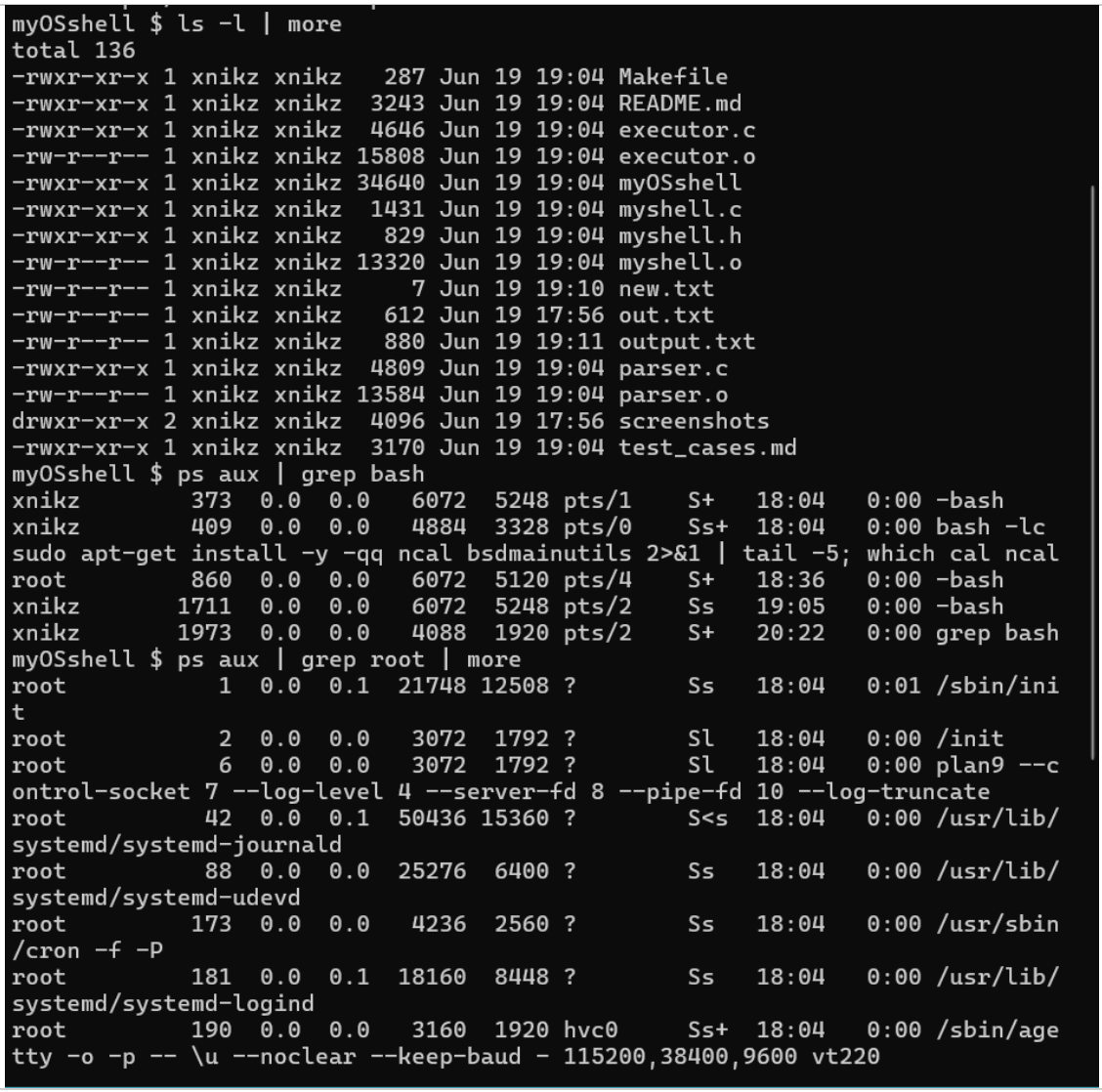
```
myOSshell $ sleep 10 &
[background pid: 1967]
myOSshell $ ps
    PID TTY          TIME CMD
   1711 pts/2    00:00:00 bash
   1895 pts/2    00:00:00 myOSshell
   1967 pts/2    00:00:00 sleep
   1968 pts/2    00:00:00 ps
myOSshell $
```

圖 9-9：sleep 10 & 背景執行，立即顯示 [background pid: 1967]，ps 可看到 sleep 處理元

待數秒後 sleep 處理元自然結束，由 SIGCHLD 訊號處理機制（詳見第四章 4.4 節）自動回收，不會在系統中留下殭屍處理元。

9.6 Pipe 測試

依序測試 ls -l | more（單一 pipe）、ps aux | grep bash（單一 pipe 搭配文字過濾）、ps aux | grep root | more（雙重 pipe）。三項測試的輸出結果如下：ls -l 的長格式輸出正確接到 more；ps aux 的輸出經過 grep bash 過濾後，只留下指令名稱包含 bash 的處理元；ps aux 的輸出再經過 grep root 過濾、接到 more 分頁顯示，只留下使用者為 root 的處理元清單：



```
myOSshell $ ls -l | more
total 136
-rwxr-xr-x 1 xnikz xnikz   287 Jun 19 19:04 Makefile
-rwxr-xr-x 1 xnikz xnikz  3243 Jun 19 19:04 README.md
-rwxr-xr-x 1 xnikz xnikz  4646 Jun 19 19:04 executor.c
-rw-r--r-- 1 xnikz xnikz 15808 Jun 19 19:04 executor.o
-rwxr-xr-x 1 xnikz xnikz 34640 Jun 19 19:04 myOSshell
-rwxr-xr-x 1 xnikz xnikz  1431 Jun 19 19:04 myshell.c
-rwxr-xr-x 1 xnikz xnikz   829 Jun 19 19:04 myshell.h
-rw-r--r-- 1 xnikz xnikz 13320 Jun 19 19:04 myshell.o
-rw-r--r-- 1 xnikz xnikz     7 Jun 19 19:10 new.txt
-rw-r--r-- 1 xnikz xnikz   612 Jun 19 17:56 out.txt
-rw-r--r-- 1 xnikz xnikz   880 Jun 19 19:11 output.txt
-rwxr-xr-x 1 xnikz xnikz  4809 Jun 19 19:04 parser.c
-rw-r--r-- 1 xnikz xnikz 13584 Jun 19 19:04 parser.o
drwxr-xr-x 2 xnikz xnikz  4096 Jun 19 17:56 screenshots
-rwxr-xr-x 1 xnikz xnikz  3170 Jun 19 19:04 test_cases.md
myOSshell $ ps aux | grep bash
xnikz        373  0.0  0.0   6072  5248 pts/1    S+   18:04   0:00 -bash
xnikz        409  0.0  0.0   4884  3328 pts/0    Ss+  18:04   0:00 bash -lc
sudo apt-get install -y -qq ncal bsdmainutils 2>&1 | tail -5; which cal ncal
root         860  0.0  0.0   6072  5120 pts/4    S+   18:36   0:00 -bash
xnikz       1711  0.0  0.0   6072  5248 pts/2    Ss   19:05   0:00 -bash
xnikz       1973  0.0  0.0   4088  1920 pts/2    S+   20:22   0:00 grep bash
myOSshell $ ps aux | grep root | more
root           1  0.0  0.1  21748 12508 ?        Ss   18:04   0:01 /sbin/ini
t
root           2  0.0  0.0   3072  1792 ?        Sl   18:04   0:00 /init
root           6  0.0  0.0   3072  1792 ?        Sl   18:04   0:00 plan9 --c
ontrol-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-truncate
root          42  0.0  0.1  50436 15360 ?        S<s  18:04   0:00 /usr/lib/
systemd/systemd-journald
root          88  0.0  0.0  25276  6400 ?        Ss   18:04   0:00 /usr/lib/
systemd/systemd-udevd
root         173  0.0  0.0   4236  2560 ?        Ss   18:04   0:00 /usr/sbin
/cron -f -P
root         181  0.0  0.1  18160  8448 ?        Ss   18:04   0:00 /usr/lib/
systemd/systemd-logind
root         190  0.0  0.0   3160  1920 hvc0     Ss+  18:04   0:00 /sbin/age
tty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
```

圖 9-10：依序測試 ls -l | more、ps aux | grep bash、ps aux | grep root | more

三項測試皆正確執行，證明本實作除了支援單一 pipe，也確實支援多重 pipe（兩個以上的 | 串接）。

9.7 Redirection 測試

依序測試 ls -l > output.txt（覆蓋寫入）、cat output.txt（確認內容）、echo hello >> output.txt（追加寫入）、cat output.txt（再次確認）。從下圖可見，第一次 cat 顯示的是 ls -l 的完整輸出，執行 echo hello >> output.txt 後，第二次 cat 顯示的內容在原本的檔案列表之後多了一行 hello，證明 >> 確實是「追加」而非「覆蓋」：

```
myOSshell $ ls -l > output.txt
myOSshell $ cat output.txt
total 132
-rwxr-xr-x 1 xnikz xnikz   287 Jun 19 19:04 Makefile
-rwxr-xr-x 1 xnikz xnikz  3243 Jun 19 19:04 README.md
-rwxr-xr-x 1 xnikz xnikz  4646 Jun 19 19:04 executor.c
-rw-r--r-- 1 xnikz xnikz 15808 Jun 19 19:04 executor.o
-rwxr-xr-x 1 xnikz xnikz 34640 Jun 19 19:04 myOSshell
-rwxr-xr-x 1 xnikz xnikz  1431 Jun 19 19:04 myshell.c
-rwxr-xr-x 1 xnikz xnikz   829 Jun 19 19:04 myshell.h
-rw-r--r-- 1 xnikz xnikz 13320 Jun 19 19:04 myshell.o
-rw-r--r-- 1 xnikz xnikz     7 Jun 19 19:10 new.txt
-rw-r--r-- 1 xnikz xnikz   612 Jun 19 17:56 out.txt
-rw-r--r-- 1 xnikz xnikz     0 Jun 19 20:22 output.txt
-rwxr-xr-x 1 xnikz xnikz  4809 Jun 19 19:04 parser.c
-rw-r--r-- 1 xnikz xnikz 13584 Jun 19 19:04 parser.o
drwxr-xr-x 2 xnikz xnikz  4096 Jun 19 17:56 screenshots
-rwxr-xr-x 1 xnikz xnikz  3170 Jun 19 19:04 test_cases.md
myOSshell $ echo hello >> output.txt
myOSshell $ cat output.txt
total 132
-rwxr-xr-x 1 xnikz xnikz   287 Jun 19 19:04 Makefile
-rwxr-xr-x 1 xnikz xnikz  3243 Jun 19 19:04 README.md
-rwxr-xr-x 1 xnikz xnikz  4646 Jun 19 19:04 executor.c
-rw-r--r-- 1 xnikz xnikz 15808 Jun 19 19:04 executor.o
-rwxr-xr-x 1 xnikz xnikz 34640 Jun 19 19:04 myOSshell
-rwxr-xr-x 1 xnikz xnikz  1431 Jun 19 19:04 myshell.c
-rwxr-xr-x 1 xnikz xnikz   829 Jun 19 19:04 myshell.h
-rw-r--r-- 1 xnikz xnikz 13320 Jun 19 19:04 myshell.o
-rw-r--r-- 1 xnikz xnikz     7 Jun 19 19:10 new.txt
-rw-r--r-- 1 xnikz xnikz   612 Jun 19 17:56 out.txt
-rw-r--r-- 1 xnikz xnikz     0 Jun 19 20:22 output.txt
-rwxr-xr-x 1 xnikz xnikz  4809 Jun 19 19:04 parser.c
-rw-r--r-- 1 xnikz xnikz 13584 Jun 19 19:04 parser.o
drwxr-xr-x 2 xnikz xnikz  4096 Jun 19 17:56 screenshots
-rwxr-xr-x 1 xnikz xnikz  3170 Jun 19 19:04 test_cases.md
hello
```

圖 9-11：ls -l > output.txt 覆蓋寫入，cat 確認後再以 echo hello >> output.txt 追加寫入

接著測試 cat < output.txt：cat 本身是從標準輸入讀取資料，但透過 < 將標準輸入重新導向為 output.txt 檔案後，cat 印出的內容與檔案內容完全一致（包含前一步驟追加的 hello），證明重新導向對 cat 而言是透明的：

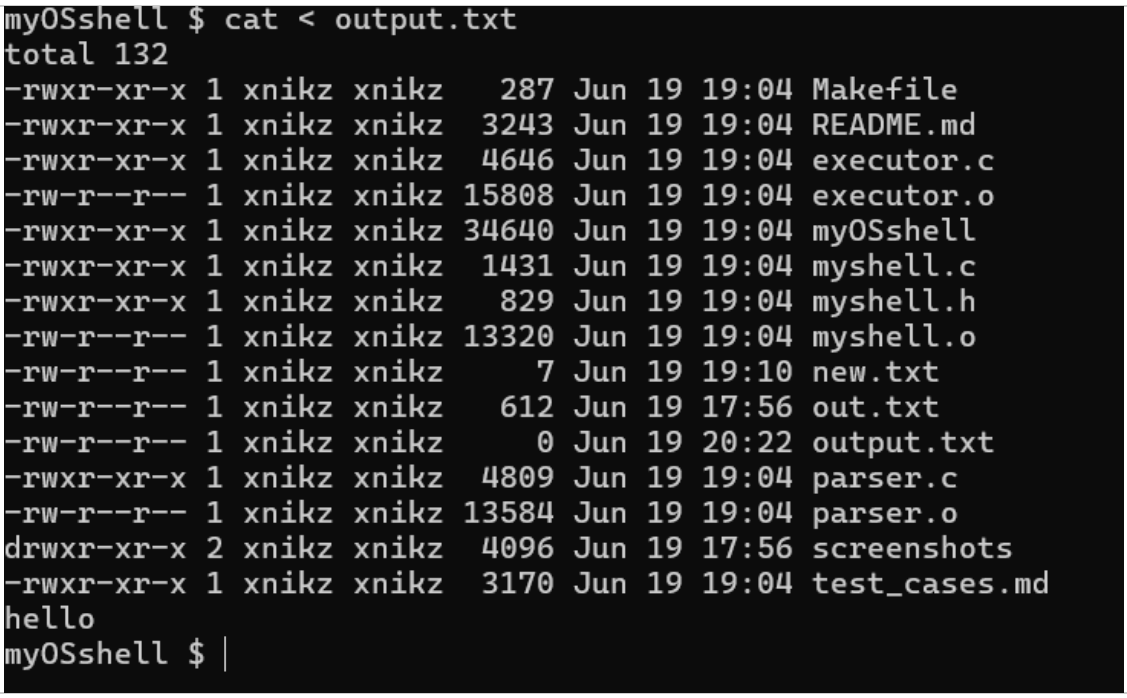


圖 9-12：cat < output.txt，標準輸入被重新導向為檔案內容，輸出包含先前追加的 hello

9.8 內建指令測試

依序測試 pwd、cd ..、pwd、cd /tmp、pwd、exit。從下圖可見，一開始 pwd 顯示 /home/xnikz/myOSshell_project，執行 cd .. 後 pwd 變成 /home/xnikz，執行 cd /tmp 後 pwd 變成 /tmp，證明 cd 確實改變了 Shell 本身（parent process）的工作目錄，而不是只改變某個短暫存在的子處理元的工作目錄；最後輸入 exit，myOSshell 正常結束並把終端機控制權交還給原本的系統 shell（提示字元變回 xnikz@johnny:~/myOSshell_project$）：

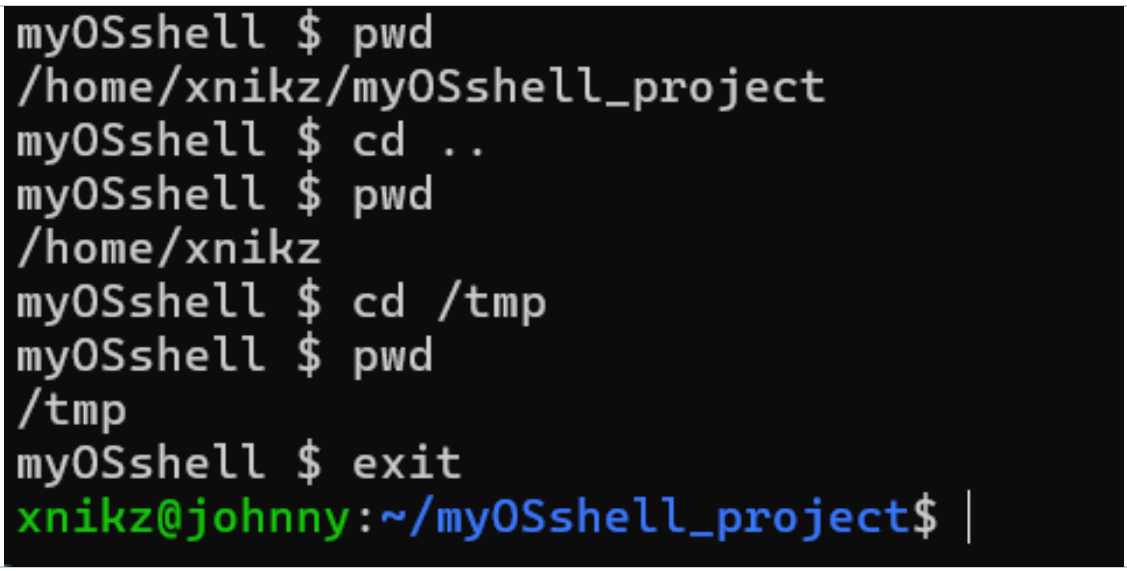


圖 9-13：依序執行 pwd、cd ..、pwd、cd /tmp、pwd、exit，驗證 cd 改變 Shell 本身工作目錄

9.9 錯誤處理測試

依序測試全部 7 種錯誤輸入情境：輸入不存在的指令 abcdefg、cd 到不存在的資料夾 nofolder、pipe 左邊缺指令 ls |、pipe 右邊缺指令 | more、輸入重新導向的檔案不存在 cat < nofile.txt、輸出重新導向缺檔名 ls >、追加輸出缺檔名 echo hello >>。下圖完整呈現所有測試結果，每一種錯誤情境都顯示對應的錯誤訊息，且 Shell 在顯示錯誤訊息後都正確回到 myOSshell $ 提示字元，沒有任何一種情況導致程式崩潰或當機：

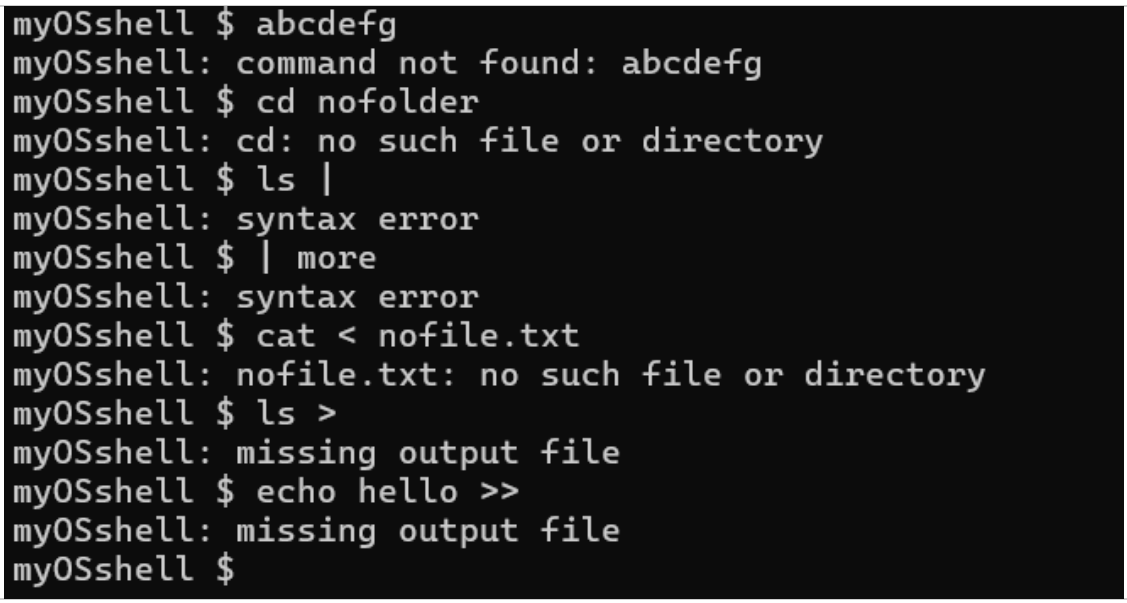


圖 9-14：依序測試 7 種錯誤輸入情境，每種情境皆正確顯示錯誤訊息並回到提示字元

以上九大類測試已於 Ubuntu（WSL2）環境下逐項手動執行驗證，涵蓋基本指令、16 項指定指令、額外建議指令、背景執行、單一與多重 pipe、輸出輸入重新導向、內建指令 cd / pwd / exit，以及全部 7 種錯誤輸入情境，myOSshell 在所有測試情境下均正常運作且未發生崩潰，完整測試紀錄詳見隨附的 test_cases.md 文件。

十、16 項指定指令功能說明

依老師要求，以下逐一說明 16 項指定指令的功能。ps 與 ls 為必須實作項目，其餘指令亦已全數完成實作與測試，所有指令皆由 myOSshell 透過 fork() 建立子處理元、再以 execvp() 呼叫對應的外部程式執行，Shell 本身並未對任何輸出內容進行攔截或寫死；實際測試畫面詳見第九章。

(1) ps：觀察系統所有的處理元資料

ps 在不加任何參數的情況下，會列出目前終端機（terminal）所附屬的處理元清單，包含 PID、TTY、執行時間（TIME）與指令名稱（CMD），用來快速確認目前 Shell 底下正在執行哪些處理元。

(2) ps axjf：連同部分處理元樹狀態顯示

ps axjf 是多個參數的組合，各參數意義如下：

- a：不與 terminal 有關的所有 process（不限於目前登入的終端機）
- u：有效使用者（effective user）相關的 process，以使用者為基準顯示詳細資訊
- x：通常與 a 這個參數一起使用，可列出較完整資訊（包含沒有控制終端機的 process）
- l：較長、較詳細地將該 PID 的資訊列出（例如 PPID、優先權、記憶體用量等）
- j：工作的格式（jobs format），著重在 PID、PGID、SID 與工作階段的關係
- f：做一個更為完整的輸出，以樹狀結構（ASCII art 形式）顯示 process 之間的父子關係

把這些參數合併成 axjf，即可在一次輸出中同時看到完整的處理元清單，並以樹狀方式呈現父處理元與子處理元之間的關係，方便觀察 myOSshell 本身與其建立的子處理元之間的層級結構。

(3) ps aux：顯示詳細 process 清單

ps aux 結合 a（所有 process）與 u（使用者導向的詳細資訊）兩個參數，x 則確保沒有控制終端

機的 process 也會被列出，輸出包含 CPU / 記憶體使用率、VSZ、RSS、STAT 狀態碼等欄位。ps aux 會依照 PID 的順序來排序顯示，方便比對同一支程式在不同時間點啟動的多個子處理元。

(4) ls -l：長格式顯示檔案資訊

ls -l 會以長格式（long listing format）列出目前目錄下的檔案與資料夾，每一行包含檔案類型與權限（permission）、連結數、擁有者、群組、檔案大小、最後修改時間與檔名，用來驗證 myOSshell 在處理一般帶參數指令時，能正確把參數透過 argv[] 傳給 execvp()。

(5) ls -l | more：pipe 串接顯示

此指令用來驗證 Pipe 管線機制：ls -l 的標準輸出會透過 pipe() + dup2() 接到 more 的標準輸入，當檔案清單超過一個畫面的長度時，more 會分頁顯示內容，藉此確認子處理元之間的資料能正確透過檔案描述符串接。

(6) top：即時系統資源監控

top 會接管整個終端機畫面，即時更新並顯示系統目前的 CPU 使用率、記憶體用量，以及依資源占用排序的處理元清單。由於 top 是一個會持續佔用終端機畫面的全螢幕程式，用來測試 myOSshell 是否能正常把終端機控制權交給子處理元；輸入 q 即可結束 top，並正確返回 myOSshell $ 提示字元。

(7) cal：顯示月曆

cal 會印出目前年月的月曆，標示出今天的日期，屬於單純讀取系統時間並輸出文字的指令，用來確認沒有任何參數、輸出內容單純的指令也能被 myOSshell 正確執行。

(8) whoami：顯示目前使用者

whoami 會印出目前登入並執行 myOSshell 的有效使用者名稱（effective user name），用來驗證子處理元繼承了 parent process（myOSshell 本身）的使用者身份與權限設定。

(9) date：顯示日期時間

date 會印出系統目前的日期與時間（包含年、月、日、星期、時、分、秒與時區），用來確認 myOSshell 執行外部指令時不會延遲或卡住，輸出的時間與實際系統時間一致。

(10) pwd：顯示工作目錄

pwd 為本專案的內建指令（builtin command），並未透過 fork() + execvp() 執行，而是直接在 myOSshell 自己的處理元中呼叫 getcwd() 取得並印出目前的工作目錄。之所以採用內建方式，是因為唯有如此才能精準反映 Shell 本身（而非某個短暫存在的子處理元）的工作目錄狀態，詳細原因已於第五章說明。

(11) mv：移動 / 重新命名檔案

mv 用來將檔案搬移到另一個位置，或是在同一個目錄下重新命名檔案。實際測試以 mv old.txt new.txt 將檔案重新命名，並以後續指令驗證新檔名確實存在、原檔名不再存在。

(12) cp：複製檔案

cp 用來複製檔案。實際測試以 cp new.txt copy.txt 複製出一份內容完全相同的新檔案，藉此驗證 myOSshell 對於帶有兩個參數（來源檔、目的檔）的外部指令也能正確解析並傳遞給 execvp()。

(13) file：判斷檔案類型

file 會檢查指定檔案的內容並判斷其檔案類型（例如純文字檔、執行檔、目錄、符號連結等）。實際測試對 copy.txt 執行 file，正確判斷出該檔案為 ASCII text（純文字檔），用來測試 myOSshell 對單一參數（檔名）指令的支援。

(14) cat：顯示檔案內容

cat 會將指定檔案的內容完整輸出到標準輸出。測試時除了單純執行 cat <檔名> 觀察畫面輸出，

也搭配第七章的 I/O Redirection 機制，測試 cat < output.txt 或搭配 >、>> 將輸出另存 / 追加到新檔案，驗證重新導向與一般外部指令可以正常組合使用。

(15) rm：刪除檔案

rm 用來刪除指定的檔案。實際測試對 copy.txt 執行 rm 之後，該檔案即從目錄中消失，藉此驗證 myOSshell 執行具破壞性（會改變檔案系統狀態）的外部指令時，行為與在一般終端機下執行完全一致。

(16) vi：執行原始代碼全螢幕編輯

vi 是一個全螢幕文字編輯器，執行後會接管整個終端機畫面，可在其中編輯原始碼或一般文字檔案。實際測試以 vi myshell.c 開啟本專案的原始碼進行編輯，確認可正常進入編輯模式；再輸入 :q 離開編輯器後，myOSshell 能正確把終端機控制權交還給自己並顯示下一次的提示字元，行為與直接在系統終端機下執行 vi 相同。

十一、開發過程中遇到的困難與解決方式

11.1 指令切割與參數解析

使用者輸入是一整行未經處理的字串，必須自行寫程式拆解出指令名稱、參數、以及 |、<、>、>>、& 等特殊符號，而不能單純依賴空白切割，因為符號與檔名之間可能沒有空白（例如 ls>output.txt）。解決方式是逐字元掃描，遇到特殊符號就先處理該符號的邏輯，再繼續往下掃描一般的參數字元，並在每個指令段各自獨立解析，最後再依 | 把多個指令段組合起來。

11.2 fork() 後 parent 與 child 的差異

剛開始實作時容易混淆「fork() 之後到底是哪一個處理元在執行哪一段程式碼」。後來理解的關鍵是：fork() 呼叫一次，但會「返回兩次」——一次在原本的處理元（parent，回傳子處理元 PID），一次在新建立的處理元（child，回傳 0）。寫程式時必須明確用 if (pid == 0) { ... } else { ... } 區分兩條執行路徑，child 路徑負責 execvp()，parent 路徑負責 waitpid() 或背景處理。

11.3 為什麼 cd 不能只用 execvp()

如前述第五章所說明，子處理元的狀態（包含工作目錄）在子處理元結束後就會消失，不會影響 parent process。這也是本專案中最直接體會到「處理元彼此獨立」這個作業系統概念的地方。

11.4 Pipe 的檔案描述符管理

實作 pipe 時最容易忽略的部分是「沒用到的檔案描述符要記得關閉」。如果 parent process 沒有把管線兩端都關閉，more 這類指令會因為讀取端永遠不會收到 EOF（因為 parent process 還握著寫入端）而一直等待，無法正常結束。解決方式是在每個 child process dup2() 完成後，立即把所有原始的管線檔案描述符全部關閉，parent process 在建立完所有 child process 之後，也統一把所有管線描述符關閉一次。

11.5 背景執行需避免殭屍處理元

如第 4.4 節所述，背景執行的子處理元結束時不會被自動回收，因此額外加上 SIGCHLD 訊號處理函式，搭配 waitpid(-1, &status, WNOHANG) 做非阻塞式的回收。

11.6 Redirection 需要 open() 與 dup2() 搭配使用

一開始嘗試直接把 printf 的輸出導向檔案，後來理解到正確的做法是在 child process 內、呼叫 execvp() 之前，用 dup2() 把標準輸入 / 輸出的檔案描述符替換掉，這樣 execvp() 執行的外部程式完全不需要修改，就能讓它的輸出寫進檔案、或讀取從檔案來的輸入。

十二、結論

透過這次 myOSshell 的實作，具體理解了以下幾個作業系統核心概念：

1. Shell 是使用者與作業系統互動的命令列介面：它本身不是指令，而是負責讀取使用者輸入、解析、並請作業系統建立新的處理元去執行指令的程式。
2. 指令執行的背後是處理元的建立與程式的載入：fork() 負責「複製出一個新的執行環境」，execvp() 負責「把這個執行環境換成想要執行的程式」，兩者搭配才能在不影響 Shell 本身的前提下執行外部指令。
3. waitpid() 是父子處理元之間的同步機制：它讓 parent process 可以選擇「等待」或「不等待」子處理元結束，這個選擇就是前景執行與背景執行的根本差異。
4. 背景執行讓父子處理元得以並行運作，但也帶來了殭屍處理元的管理問題，需要透過訊號處理機制（SIGCHLD）妥善回收。
5. Pipe 與 Redirection 本質上都是檔案描述符的操作：管線是讓兩個處理元共享同一個核心緩衝區，重新導向是讓一個處理元的標準輸入 / 輸出指向某個檔案，兩者都是透過 dup2() 達成「替換檔案描述符所指向的目標」，而不需要修改被執行的程式本身。

整體而言，這個專案讓「作業系統管理處理元」這個原本抽象的概念，變成了可以實際操作、觀察、並用截圖證明運作結果的具體經驗。

十三、附錄：使用到的系統呼叫一覽

系統呼叫	用途
fork()	建立子處理元
execvp()	在子處理元中執行外部指令
waitpid()	父處理元等待子處理元結束（前景執行）或非阻塞式回收（背景執行）
chdir()	實作內建指令 cd，改變 Shell 本身的工作目錄
getcwd()	實作內建指令 pwd，取得目前工作目錄
pipe()	建立管線，串接兩個處理元的輸入輸出
dup2()	重新導向標準輸入 / 輸出至管線或檔案
open()	開啟檔案以進行 <、>、>> 重新導向
close()	關閉不再需要的檔案描述符，避免資源洩漏與死鎖
sigaction() / SIGCHLD	監聽子處理元結束事件，回收背景處理元避免殭屍處理元